HW #4.3: Reproducible Data Analysis

Evan Gill

2026-04-14

Busiest Airports Analysis

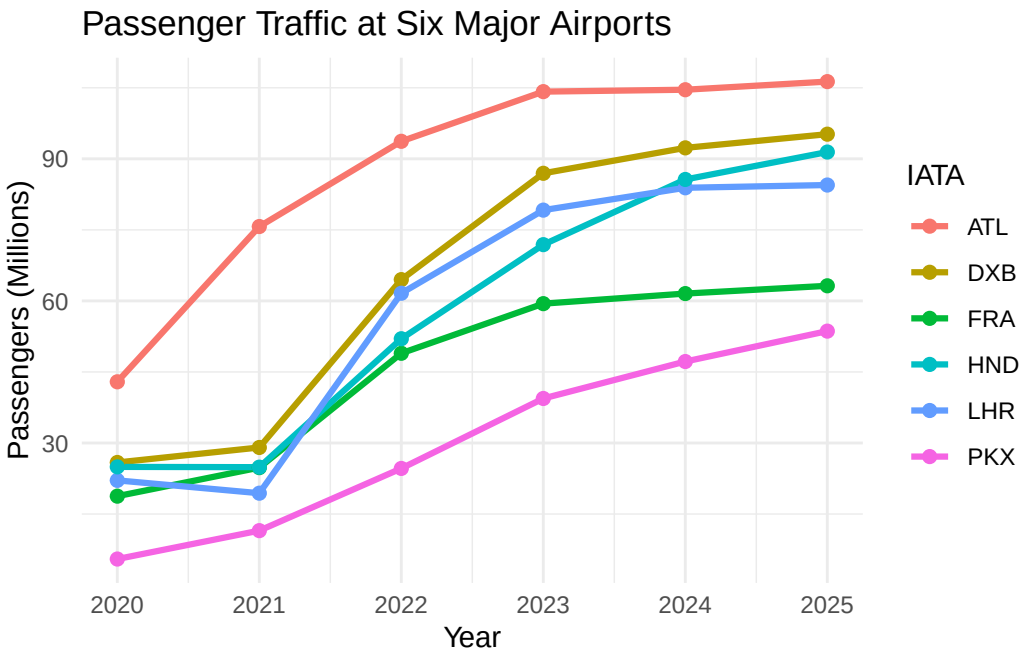
Overview

This section compares six major passenger airports from 2020 through 2025. The years 2020 and 2021 show the major negative effects caused by the COVID-19 pandemic, while later years show how airports recovered in different ways. I selected ATL, FRA, PKX, DXB, LHR, and HND because they are in different regions and had different recovery patterns.

Busiest Passenger Airports (2020–2025)

Passengers in millions

airport	iata	2020	2021	2022	2023	2024	2025
Hartsfield-Jackson Atlanta Intl (ATL)	ATL	42.92	75.70	93.70	104.19	104.59	106.30
Frankfurt Airport (FRA)	FRA	18.77	24.80	48.94	59.42	61.55	63.19
Beijing Daxing Intl (PKX)	PKX	5.50	11.49	24.63	39.40	47.20	53.62
Dubai International (DXB)	DXB	25.89	29.05	64.49	86.94	92.31	95.20
Heathrow Airport (LHR)	LHR	22.11	19.40	61.61	79.17	83.88	84.46
Tokyo Haneda (HND)	HND	24.95	24.90	52.01	71.88	85.64	91.43



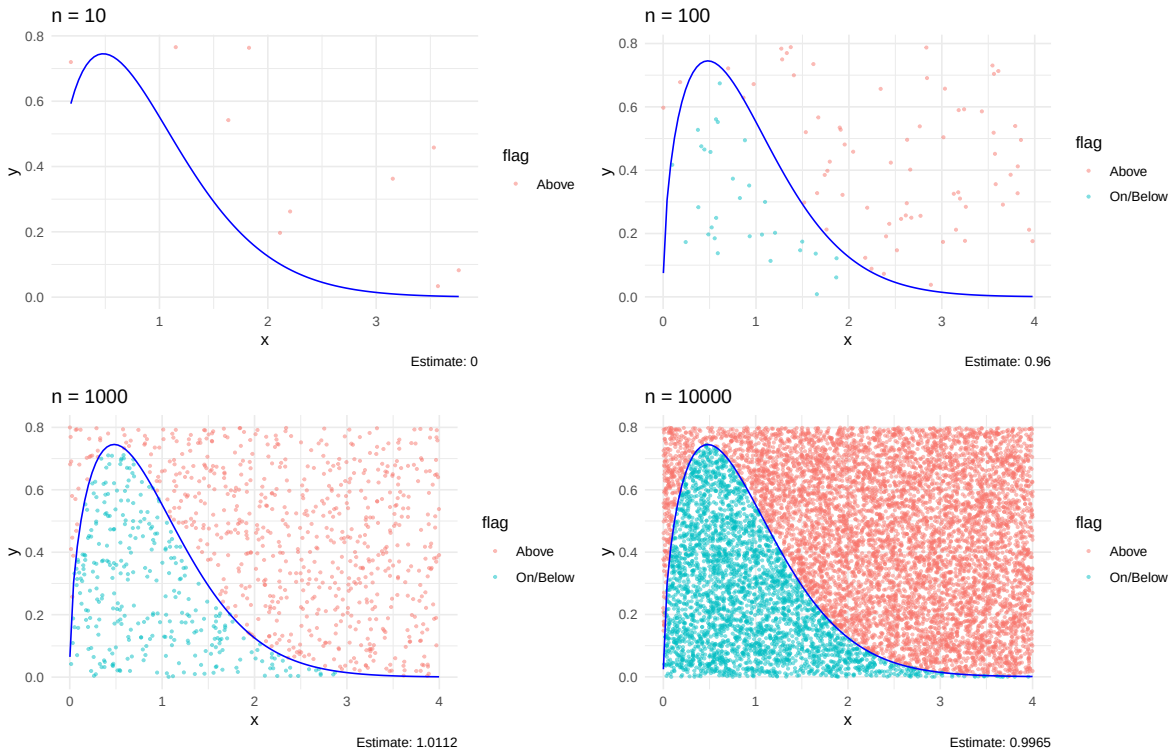
Interpretation

The table provides exact values while the line graph makes the overall trends easier to compare between. Atlanta remained the strongest performer and is over 106 million passengers by 2025. Dubai, Tokyo Haneda, and Heathrow also had strong recoveries and ended close together near the 85 to 95 million range. Frankfurt recovered more slowly than the others. Beijing Daxing is different because it opened recently, so its pattern is less about recovery and more about rapid growth from a new base. Together, the visuals show that global air travel demand returned strongly after the pandemic, but recovery speed was different depending on the airport and region.

Monte Carlo Numerical Integration

Overview

Monte Carlo integration estimates area using random sampling. For this assignment, I used the Weibull probability density function on the interval x from 0 to 4 with y from 0 to 0.8. Because the Weibull density integrates to 1 over its full support, the estimate should get closer to 1 as the number of simulated points increases.



Interpretation

The small multiple shows how simulation accuracy improves with more random points. At n = 10, the estimate can vary a lot because only a few points determine the result. By n = 1000 and n = 10000, the sampled points closely follow the area under the curve and the estimates stay near 1. This shows an important principle of simulation which is that larger sample sizes usually reduce random error and produce more reliable estimates.

Planning and Prompting GenAI Tools

Data Context and Plan

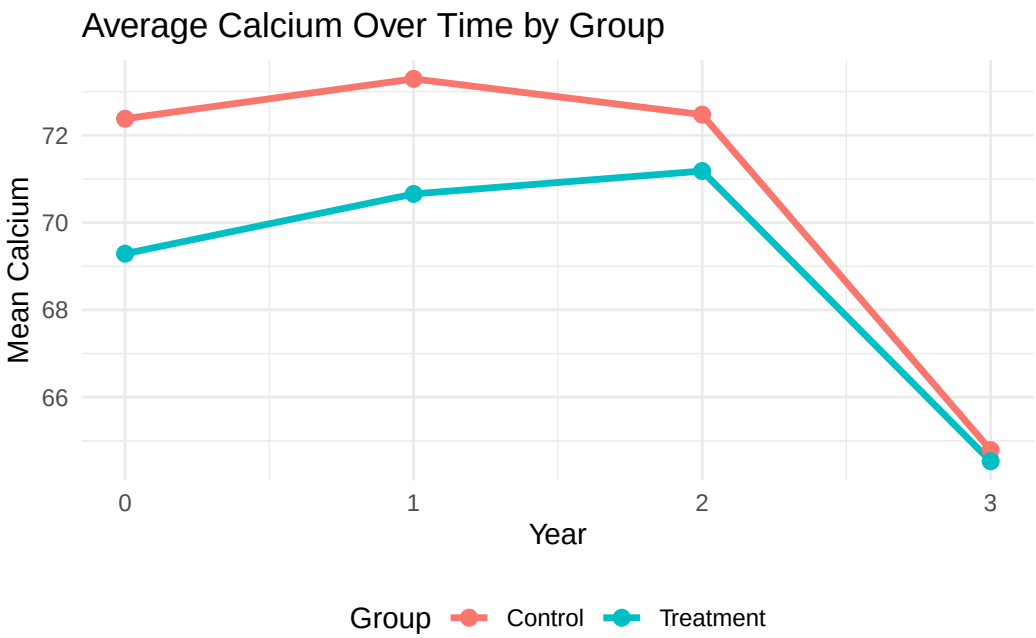
The calcium dataset follows women across four measurement periods in two groups: Control and Treatment. Because the file starts in wide format, the key task is to reorganize it into tidy form so each row represents one observation. My plan was to label the groups, assign the four time points, calculate average calcium values for each group at each year, and then create a line graph that shows changes over time.

Plan-Informed Prompt Response

For the allowed GenAI portion of the assignment, I used a prompt that explained the dataset structure and the analysis goal. That type of prompt leads to a more useful response because it knows that the data must be reshaped before graphing. A strong response recommends pivoting the data longer, creating group and year variables, summarizing means, and using a two-line comparison plot.

Generic Prompt Response

A generic prompt such as asking for a graph without context would likely produce a worse result. The generic output summarizes columns separately, but it does not use the dataset structure to compare treatment groups over time. It may suggest a chart, but it would be less likely to correctly identify the repeated measurements, the two-group structure, or the need for tidying first.



Comparison Narrative

The plan-informed response is stronger because it uses the actual structure of the dataset to guide the analysis. It identifies the need to tidy the file, preserve the two groups, and compare average calcium levels across time. The generic response is more basic because it focuses on making a graph without first solving the data organization problem. This comparison shows

that better prompts lead to better outputs when they clearly describe the data, context, and goal.

Reflection

Throughout this course, I learned that strong data analysis is more than writing code. It is understanding the context, organizing data correctly, and choosing visuals that help people learn something meaningful. Earlier in the semester, I often focused only on getting code to run. Now I think more about readability, reproducibility, and communication. For example, in the airport section I learned that a table and a graph serve different purposes. One gives precision while the other shows trends. In the Monte Carlo section, I learned how randomness can still produce accurate answers when sample sizes are large enough. I also improved at planning code before writing syntax, which saves time and reduces mistakes. Maybe the most important thing I learned is that good analysis should be understandable to another person, not just the person who wrote it. I also learned that small presentation choices matter, such as labeling figures clearly and organizing sections so a reader can follow the work. Those details make analysis more professional and easier to review.

GenAI Usage Appendix

GenAI was used only in ways permitted by the course policy.

Tool Used

ChatGPT (OpenAI GPT-5.3)

Date Used

2026-04-14

Prompts

1. Improve the grammar and clarity of my written narrative while keeping my ideas, meaning, and overall content the same. Do not change the analysis itself.
2. Create alt text and long descriptions for my airport plot and Monte Carlo simulation plot. Describe the main patterns, comparisons, and trends shown in each visual.

3. Using this calcium dataset, suggest a tidy-data workflow and a good visualization for comparing results between groups over time. Explain how to restructure the data and summarize it before plotting.
4. Answer questions about formatting my final Quarto report. Explain how to render to PDF, keep code out of the main document, and organize the GenAI Usage Appendix and Code Appendix.

Responses

1. Revise wording for grammar, sentence flow, and clarity while keeping the original ideas unchanged. Improve readability without changing the meaning or conclusions.
2. Provide alt text and longer descriptions that summarize what each chart shows. Explain overall trends, major differences between categories, and notable patterns visible in the figures.
3. Convert the calcium dataset from wide format to long format. Create variables for group and year, calculate average calcium values for each group at each time point, and build a two-line graph comparing Control and Treatment over time.
4. Render the Quarto file to PDF using the Render button in RStudio or Quarto commands. Keep code hidden from the main report, place all code in a Code Appendix, and include prompts, responses, dates, and tool details in the GenAI Usage Appendix.

Response Summary

The tool returned grammar suggestions, accessibility text for visuals, suggestions for the Planning and Prompting GenAI section, and formatting guidance.

Code Appendix

Main Packages

```
library(tidyverse)
library(gt)
```

Key Functions Used

```
## Busiest Airports: Data + Table

# create airport dataset with yearly passenger totals
airport_data <- tibble(
 airport = rep(c("Hartsfield-Jackson Atlanta Intl (ATL)","Frankfurt Airport (FRA)","Beijing I
 iata = rep(c("ATL","FRA","PKX","DXB","LHR","HND"), each=6),
 year = rep(2020:2025, times=6),
 passengers = c(42918685,75704760,93699954,104187868,104586285,106302208,18774447,24799464,48

# convert passengers to millions and reshape for table display
 airport_data |>
 mutate(passengers_millions = round(passengers/1e6,2)) |>
 select(airport,iata,year,passengers_millions) |>
 pivot_wider(names_from=year, values_from=passengers_millions) |>
 gt() |>
 tab_header(title="Busiest Passenger Airports (2020-2025)", subtitle="Passengers in millions"


## Busiest Airports: Visualization

# plot passenger traffic trends over time
airport_data |>
 mutate(passengers_millions = passengers/1e6) |>
 ggplot(aes(year, passengers_millions, color=iata, group=iata))+
 geom_line(linewidth=1.1)+geom_point(size=2)+
 scale_x_continuous(breaks=2020:2025)+
 labs(title="Passenger Traffic at Six Major Airports", x="Year", y="Passengers (Millions)", c
 theme_minimal()


## Monte Carlo: Helper Function

# generate random points inside the bounding rectangle
run_mc_simulation <- function(n,x_min,x_max,y_min,y_max){
 tibble(x=runif(n,x_min,x_max), y=runif(n,y_min,y_max))
}


## Monte Carlo: Simulation and Small Multiple
```

```
#| layout-ncol: 2

# create one Monte Carlo plot and estimate area under the curve
make_plot <- function(n){
 d <- run_mc_simulation(n,0,4,0,0.8) |>
 mutate(flag = if_else(y <= dweibull(x, shape=1.5, scale=1), "On/Below","Above"))
 est <- mean(d$flag=="On/Below") * (4*0.8)
 ggplot(d,aes(x,y,color=flag))+
 geom_point(alpha=.5,size=.6)+
 stat_function(fun=dweibull,args=list(shape=1.5,scale=1),color="blue")+
 labs(title=paste0("n = ",n), caption=paste("Estimate:", round(est,4)))+
 theme_minimal()
}

set.seed(123)
make_plot(10)
make_plot(100)
make_plot(1000)
make_plot(10000)


## Planning & Prompting: Calcium Wrangling

# read original dataset
calcium <- read.csv("calcium.csv")

# store column names for group mapping
col_names <- names(calcium)

# convert data to long format and assign group/year labels
long <- calcium |>
  mutate(id = row_number()) |>
  pivot_longer(cols = -id, names_to = "var", values_to = "calcium") |>
  mutate(
    col_num = match(var, col_names),
    group = case_when(
      col_num %in% 1:4 ~ "Control",
      col_num %in% 5:8 ~ "Treatment"
    ),
    year = rep(0:3, times = n()/4)
  )

# compute average calcium by group and year
```

```
summary_df <- long |>
  group_by(group, year) |>
  summarise(mean_calcium = mean(calcium, na.rm = TRUE), .groups = "drop")


## Planning & Prompting: Calcium Visualization

# plot average calcium over time for both groups
ggplot(summary_df, aes(year, mean_calcium, color = group, group = group)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = 0:3) +
  labs(
    title = "Average Calcium Over Time by Group",
    x = "Year",
    y = "Mean Calcium",
    color = "Group"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```